字符串哈希
字典树
KMP
KMP自动机
拓展KMP(Z函数)
AC自动机(优化后)
后缀数组
后缀自动机
$Manacher$

字符串

字符串哈希

```
ull p[N];
void init()
{
    p[0] = 1;
    for(int i = 1;i < N;i++)
        p[i] = p[i - 1] * 131;
}
ull get_ha(std::string &s)//获取某一个字符串的哈希值
{
    int n = s.size();
    ull ha = 0;
    for(int i = 1;i <= n;i++)
        ha = ha * 131 + s[i - 1];
    return ha;
}
std::vector<ull>init_hash(std::string &s)//构造s的字符串哈希
{
    int n = s.size();
    std::vector<ull>ha(n + 1);
    for(int i = 1;i <= n;i++)
        ha[i] = ha[i - 1] * 131 + s[i - 1];
    return ha;
}
ull get_hash(std::vector<ull> &ha,int l,int r)//获取子串的哈希值
{
    return ha[r] - ha[l - 1] * p[r - l + 1];
}
```

```
constexpr int P = 1e9 + 7;
//双值哈希
ull p1[N],p2[N];
void init()
{
    p2[0] = p1[0] = 1;
    for(int i = 1;i < N;i++)
```

```
    {
        p1[i] = p1[i - 1] * 131;
        p2[i] = p2[i - 1] * 131 % P;
    }
}
std::vector<ull>init_hash1(std::string &s)
{
    int n = s.size();
    std::vector<ull>ha(n + 1);
    for(int i = 1;i <= n;i++)
        ha[i] = ha[i - 1] * 131 + s[i - 1];
    return ha;
}
std::vector<ull>init_hash2(std::string &s)
{
    int n = s.size();
    std::vector<ull>ha(n + 1);
    for(int i = 1;i <= n;i++)
        ha[i] = ((ha[i - 1] * 131) % P + s[i - 1]) % P;
    return ha;
}
ull get_hash1(std::vector<ull> &ha,int l,int r)
{
    return ha[r] - ha[l - 1] * p1[r - l + 1];
}
ull get_hash2(std::vector<ull> &ha,int l,int r)
{
    return (ha[r] - (ha[l - 1] * p2[r - l + 1]) % P + P) % P;
}
ull get_hash1(std::string &s)
{
    ull ha = 0;
    for(char c : s)
        ha = ha * 131 + c;
    return ha;
}
ull get_hash2(std::string &s)
{
    ull ha = 0;
    for(char c : s)
        ha = (ha * 131 % P + c) % P;
    return ha;
}
```

字典树

```
struct trie
{
    int n,tot;
    std::vector<std::vector<int>>nx;
```

```
    std::vector<int>cnt;
    trie(int n,int m = 26)
    {
        n = n;
        tot = 0;
        nx.resize(n + 1,std::vector<int>(m));
        cnt.resize(n + 1);
    }
    int insert(std::string &s)
    {
        int pos = 0;
        for(char c : s)
        {
            int j = c - 'a';
            if(!nx[pos][j])nx[pos][j] = ++tot;
            pos = nx[pos][j];
        }
        cnt[pos]++;
        return pos;
    }
    bool find(std::string &s)
    {
        int pos = 0;
        for(char c : s)
        {
            int j = c - 'a';
            if(!nx[pos][j])return 0;
            pos = nx[pos][j];
        }
        return cnt[pos];
    }
};
```

KMP

```
std::vector<int> KMP(std::string &s)
{
    int n = s.size();
    std::vector<int>pi(n);
    for(int i = 1;i < n;i++)
    {
        int j = pi[i - 1];
        while(j > 0 && s[i] != s[j])j = pi[j - 1];
        if(s[i] == s[j])j++;
        pi[i] = j;
    }
    return pi;
}
```

```
void KMP(string s,string t)
```

```
{
    int n = s.size();
    int m = t.size();
    vector<int>pi(1 + n);//s + "#"的前缀函数数组
    s = s + '#';
    for(int i = 1;i <= n;i++)
    {
        int j = pi[i - 1];
        while(j > 0 && s[i] != s[j])j = pi[j - 1];
        if(s[i] == s[j])j++;
        pi[i] = j;
    }

    vector<int>ans;//s在t中出现的位置
    int la = 0;
    for(int i = 0;i < m;i++)
    {
        int j = la;
        while(j > 0 && t[i] != s[j])j = pi[j - 1];
        if(t[i] == s[j])j++;
        if(j == n)ans.push_back(i - n + 2);
        la = j;
    }
    for(int v : ans)cout << v << endl;
    for(int i = 0;i < n;i++)
        cout << pi[i] << " ";
    cout << endl;
}


//统计每个前缀在同一个字符串中出现的次数
vector<int> ans(n + 1);
for (int i = 0; i < n; i++) ans[pi[i]]++;
for (int i = n - 1; i > 0; i--) ans[pi[i - 1]] += ans[i];
for (int i = 0; i <= n; i++) ans[i]++;

//统计每个前缀在另一个字符串t中出现的次数
//与上文相似，我们只需要统计t中的前缀函数值，然后叠加即可
vector<int>ans(n + 1);
int la = 0;
for(int i = 0;i < m;i++)
{
    int j = la;
    while(j > 0 && t[i] != s[j])j = pi[j - 1];
    if(t[i] == s[j])j++;
    if(j == n)ans.push_back(i - n + 2);
    la = j;
    ans[la]++;统计
}
for (int i = n - 1; i > 0; i--) ans[pi[i - 1]] += ans[i];叠加
```

KMP自动机

```
void KMP_auto(string s,string t)
{
    int n = s.size();
    int m = t.size();
    std::vector aut(n + 1,std::vector<int>(26));//自动机
    std::vector<int>pi(n + 1);
    s = s + '#';
    for(int i = 0;i <= n;i++)
    {
        for(int c = 0;c < 26;c++)
        {
            if(i > 0 && s[i] - 'A' != c)
                aut[i][c] = aut[pi[i - 1]][c];
            else
                aut[i][c] = i + (s[i] - 'A' == c);
        }
        if(i > 0)pi[i] = aut[pi[i - 1]][s[i] - 'A'];
    }
    std::vector<int>ans;
    int la = 0;
    for(int i = 0;i < m;i++)
    {
        int j = aut[la][t[i] - 'A'];
        if(j == n)ans.push_back(i - n + 2);
        la = j;
    }
    for(int v : ans)
        cout << v << endl;
    for(int i = 0 ;i < n;i++)
        cout << pi[i] << " ";
    cout << endl;
}
```

拓展KMP(Z函数)

```
std::vector<int> Zf(std::string &s)
{
    int n = s.size();
    std::vector<int>z(n);//LCP(s,s[i..n - 1])(最长公共前缀)
    for(int i = 1,l = 0,r = 0;i < n;i++)//[l,r)当前处理区间
    {
        if(i < r)
            z[i] = std::min(r - i,z[i - l]);
        while(i + z[i] < n && s[z[i]] == s[i + z[i]])
            z[i]++;
        if(i + z[i] > r)
        {
            l = i;
            r = i + z[i];
```

```
        }
    }
    return z;
}
```

AC自动机(优化后)

```
struct Graph
{
    struct edge
    {
        int nx;
        int to;
        int w;
    };
    std::vector<edge>e;
    std::vector<int>heads;
    Graph(int n = 0,int m = 0) : heads(n + 1,-1)
    {
        e.reserve(m);
    }
    void add(int u,int v,int w = 0)
    {
        e.push_back({heads[u],v,w});
        heads[u] = e.size() - 1;
    }
};
struct AC : Graph
{
    struct node
    {
        std::vector<int>son;
        int fail;
        int cnt;
        int idx;
        void init(int m = 26)
        {
            son.resize(m);
            fail = cnt = idx = 0;
        }
    };
    int tot,pidx;
    std::vector<node>tr;
    AC(int n): Graph(n),tr(n + 1)
    {
        tot = pidx = 0;
        tr[0].init();
    }
    int insert(std::string &s)
    {
        int pos = 0;
        for(char c : s)
```

```
        {
            int j = c - 'a';
            int &son = tr[pos].son[j];
            if(!son)son = ++tot,tr[son].init();
            pos = son;
        }
        if(!tr[pos].idx)tr[pos].idx = ++pidx;
        return tr[pos].idx;
    }
    void build(int m = 26)
    {
        std::queue<int>q;
        for(int i = 0;i < m;i++)
        {
            int son = tr[0].son[i];
            if(son)q.push(son),add(0,son);
        }
        while(!q.empty())
        {
            int u = q.front();q.pop();
            for(int i = 0;i < m;i++)
            {
                if(tr[u].son[i])
                {
                    tr[tr[u].son[i]].fail = tr[tr[u].fail].son[i];
                    add(tr[tr[u].fail].son[i],tr[u].son[i]);
                    q.push(tr[u].son[i]);
                }
                else tr[u].son[i] = tr[tr[u].fail].son[i];
            }
        }
    }
    void dfs(int u)
    {
        for(int i = heads[u];~i;i = e[i].nx)
        {
            int v = e[i].to;
            dfs(v);
            tr[u].cnt += tr[v].cnt;
        }
    }
    std::vector<int> query(std::string &s)
    {
        std::vector<int>ans(pidx + 1);
        int pos = 0;
        for(char c : s)
        {
            int j = c - 'a';
            pos = tr[pos].son[j];
            tr[pos].cnt++;
        }
        dfs(0);
        for(int i = 0;i <= tot;i++)
```

```
            ans[tr[i].idx] += tr[i].cnt;
        return ans;
    }
};
void solve()
{
    int n,sum = 0;std::cin >> n;
    std::vector<std::string>a(n);
    std::vector<int>idx(n);
    for(int i = 0;i < n;i++)
    {
        std::cin >> a[i];
        sum += a[i].size();
    }
    AC ac(sum);
    for(int i = 0;i < n;i++)
        idx[i] = ac.insert(a[i]);
    ac.build();
    std::string s;std::cin >> s;
    auto ans = ac.query(s);
    for(int i = 0;i < n;i++)
        std::cout << ans[idx[i]] << "\n";
}
```

后缀数组

```
lcp(sa[i],sa[j]) = min{ht[i + 1],...ht[j]};
ht[i] = lcp(sa[i],sa[i - 1]);
//lcp(i,j)代表后缀i和后缀j的最长公共后缀
struct SA
{
    string s;
    int n,sa[N],osa[N],rk[N],ork[N],cnt[N],ht[N],st_mn[N][21];
    int lcp(int l,int r)
    {
        l = rk[l],r = rk[r];
        if(l > r)swap(l,r);
        int k = log2(r - l + 1);
        return min(st_mn[l][k],st_mn[r - (1LL << k) + 1][k]);
    }
    void init(string t)
    {
        memset(cnt,0,sizeof(cnt));
        int n = t.size();
        s = " " + t;
        int m = 128,p;
        for(int i = 1;i <= n;i++)cnt[rk[i] = s[i]]++;
        for(int i = 1;i <= m;i++)cnt[i] += cnt[i - 1];
        for(int i = n;i >= 1;i--)sa[cnt[rk[i]]--] = i;
        for(int w = 1;;w <<= 1,m = p)
        {
            int cur = 0;
```

```
            for(int i = n - w + 1;i <= n;i++)osa[++cur] = i;
            for(int i = 1;i <= n;i++)
                if(sa[i] > w)osa[++cur] = sa[i] - w;
            memset(cnt,0,sizeof(int) * (m + 1));
            for(int i = 1;i <= n;i++)cnt[rk[osa[i]]]++;
            for(int i = 1;i <= m;i++)cnt[i] += cnt[i - 1];
            for(int i = n;i >= 1;i--)sa[cnt[rk[osa[i]]]--] = osa[i];
            memcpy(ork,rk,sizeof(int) * (n + 1));
            p = 0;
            for(int i = 1;i <= n;i++)
            {
                if(ork[sa[i]] == ork[sa[i - 1]] && ork[sa[i] + w] == ork[sa[i - 1] +
w])
                    rk[sa[i]] = p;
                else rk[sa[i]] = ++p;
            }
            if(p == n)break;
        }
        for(int i = 1,k = 0;i <= n;i++)
        {
            if(k)k--;
            while(s[sa[rk[i]] + k] == s[sa[rk[i] - 1] + k])k++;
            ht[rk[i]] = k;
        }
        for(int i = 1;i <= n;i++)st_mn[i][0] = ht[i];
        for(int j = 1;j < 21;j++)
            for(int i = 1;i + (1LL << j) - 1 <= n;i++)
                st_mn[i][j] = min(st_mn[i][j - 1],st_mn[i + (1LL << j - 1)][j - 1]);
    }
}sa;
```

后缀自动机

```
/*
    对于u的权值为c的nx边，指向的v，将u的endpos集合 + 1，字符为c的集合传递给v。
    对于根节点，包含所有的endpos
*/
struct SAM
{
    struct state
    {
        int len,link;//当前节点代表的最长的子串，指向的最长后缀
        map<char,int> nx;//指向下一个状态
        void init()
        {
            len = 0;
            link = -1;
            nx.clear();
        }
    }st[N << 1];//节点数量最多2n - 1
    int tot,la;//当前节点数量和当前最长的前缀节点
    void init()
```

```
    {
        tot = la = 0;
        st[0].init();
    }
    void sam_ex(char c)//在线构造后缀自动机
    {
        int cur = ++tot;st[cur].init();
        st[cur].len = st[la].len + 1;//前缀长度为最长子串
        int p = la;
        for(;p != -1 && !st[p].nx.count(c);p = st[p].link)st[p].nx[c] = cur;
        if(p == -1)st[cur].link = 0;
        else
        {
            int q = st[p].nx[c];
            /*
                len(p) + 1是当前字符串最长的一个后缀
                如果这个后缀小于 q 的最长字符串s，说明这个后缀的
                等价位增加了(也就是当前字符串末尾)，且s没有增加
            */
            if(st[p].len + 1 == st[q].len)st[cur].link = q;
            else
            {
                //复制，注意，根据题目维护的其他值不一定相等，需要重新赋值
                int cln = ++tot;st[cln] = st[q];
                //下面的最长子串长度就是例子之一
                st[cln].len = st[p].len + 1;
                for(;p != -1 && st[p].nx.count(c) && st[p].nx[c] == q;p =
st[p].link)st[p].nx[c] = cln;
                st[q].link = st[cur].link = cln;
            }
        }
        la = cur;
    }
}sam;
void solve()
{
    string s;
    cin >> s;
    sam.init();
    for(char c : s)sam.sam_ex(c);
}
```

Manacher

```
//[l,r)区间找中心点，然后计算对称点
std::vector<int> Manacher_odd(std::string &s)
{
    int n = s.size();
    std::vector<int>d1(n,1);//d1[i]: [i - d1[i] + 1,i + d1[i])
    for(int i = 0,l = 0,r = 0;i < n;i++)
    {
        if(i < r)
```

```
            d1[i] = std::min(r - i,d1[l + r - 1 - i]);
        while(i - d1[i] >= 0 && i + d1[i] < n && s[i - d1[i]] == s[i + d1[i]])
            d1[i]++;
        if(i + d1[i] > r)
        {
            l = i - d1[i] + 1;
            r = i + d1[i];
        }
    }
    return d1;
}
std::vector<int> Manacher_even(std::string &s)
{
    int n = s.size();

    std::vector<int>d2(n);//d2[i]: [i - d2[i],i + d2[i])
    for(int i = 0,l = 0,r = 0;i < n;i++)
    {
        if(i < r)
            d2[i] = std::min(r - i,d2[l + r - i]);
        while(i - 1 - d2[i] >= 0 && i + d2[i] < n && s[i - 1 - d2[i]] == s[i + d2[i]])
            d2[i]++;
        if(i + d2[i] > r)
        {
            l = i - d2[i];
            r = i + d2[i];
        }
    }
    return d2;
}
```